

Interpretable Rotor-Bar Fault Diagnosis of Squirrel-Cage Induction Motors Based on Polynomial Temporal Convolutional Network

APPENDIX

This supplementary appendix provides validation, implementation, configuration, and measurement details that are not fully presented in the main manuscript due to space limitations. All supplementary analyses correspond to the same squirrel-cage induction motor (SCIM) rotor-bar fault-diagnosis task, in which the phase-C open-circuit induced-voltage response is used as the diagnostic signal. The experimental scope remains controlled pre-assembly rotor-quality inspection using the seeded-defect rotor specimens described in the main manuscript. The material below supplements complete-recording-wise validation, stochastic training variability, polynomial-representation comparisons, polynomial-order selection, noise sensitivity, implementation settings, comparison-model configurations, and edge-platform measurement protocols.

The supplementary evidence does not extend the experimental scope to previously unseen physical rotors. Each rotor condition contains five acquisition recordings, but each fault type and severity is represented by only one physical rotor specimen. Complete-recording-wise evaluations therefore characterize recording-level stability within the present controlled specimen set, whereas repeated training runs characterize stochastic optimization variability.

A. Supplementary Validation for Diagnostic Accuracy

The primary dataset partition was performed before signal segmentation. For each rotor condition, three complete acquisition recordings were assigned to training, one to validation, and one to the held-out test set. Non-overlapping 256-point segments were generated only after this recording assignment, ensuring no cross-recording leakage.

A supplementary five-fold evaluation was conducted using complete acquisition recordings as fold units. This design evaluates robustness to recording-level variability, i.e., whether the model remains stable when different full acquisition runs are used as unseen test sources. As shown in Table A-I, the proposed PTCN achieves $99.78 \pm 0.30\%$ mean accuracy, indicating stable performance with respect to the choice of held-out recording within the current dataset.

Stochastic variability was evaluated using four random seeds with repeated runs under identical settings. This experiment isolates the effect of optimization randomness (initialization, minibatch ordering, and gradient path variability) while keeping dataset partition fixed.

As summarized in Table A-II, the overall accuracy is $99.87 \pm 0.41\%$, indicating low sensitivity to initialization randomness under the current training protocol. The slightly higher variance

in Seed 88 is attributed to occasional convergence to a marginally different local minimum, although performance remains within a narrow high-accuracy band.

TABLE A-I.

K-FOLD VALIDATION RESULTS OF THE PROPOSED PTCN

Fold	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Fold 1	100.00	100.00	100.00	100.00
Fold 2	100.00	100.00	100.00	100.00
Fold 3	99.41	99.44	99.41	99.41
Fold 4	100.00	100.00	100.00	100.00
Fold 5	99.48	99.50	99.48	99.48
Mean $\pm$ Std	99.78 ± 0.30	99.79 ± 0.29	99.78 ± 0.30	99.78 ± 0.30

TABLE A-II.

REPEATED RANDOM-SEED TRAINING RESULTS OF THE PROPOSED PTCN.

Random seed	Accuracy (%)
42	99.95 ± 0.11
61	99.94 ± 0.12
88	99.63 ± 0.81
123	99.94 ± 0.11
Overall	99.87 ± 0.41

Fig. A1 illustrates the training accuracy and loss evolution of the proposed PTCN. The curves show rapid convergence in early updates and stable behavior thereafter. This figure is used to demonstrate optimization stability and convergence smoothness, rather than to compare generalization performance across models.

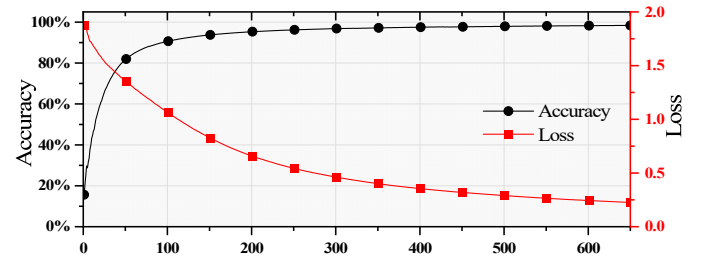


Fig. A1. Training accuracy and loss curves of the proposed PTCN.

B. Diagonal Polynomial Form vs. Full Volterra Form

The Diagonal configuration uses a compact PolyConv formulation that retains only same-index polynomial interactions within each order, effectively enforcing a structured sparsity pattern across the polynomial basis. In contrast, the Full configuration expands all local polynomial terms up to fourth order, including cross-index interactions between different temporal or feature positions.

This difference leads to a fundamental trade-off:

- a) Diagonal form: structured, interpretable, and constrained interaction space
- b) Full form: expressive, but combinatorially richer interaction space

As shown in Table A-III, the Full operator contains substantially more polynomial components and parameters, while the overall network-level parameter difference remains small due to shared downstream architecture.

The training configurations for Diagonal, Full-shared, and Full-tuned models are summarized in Table A-IV. These configurations are designed to isolate the effect of polynomial structure from optimization and regularization effects.

In particular:

- a) Diagonal model includes structural constraints (orthogonality, energy, smoothness) to stabilize compact representations.
- b) Full-shared model retains the same constraints to ensure fair comparison under identical inductive bias.
- c) Full-tuned model removes these constraints to allow unconstrained polynomial interaction learning, and is selected based on validation performance.

This design ensures that observed performance differences can be attributed primarily to representation structure rather than training imbalance.

The performance comparison under clean and noisy conditions is reported in Table A-V. This experiment evaluates robustness under Gaussian perturbation across multiple stochastic runs, rather than single-run performance.

A key observation is that the degradation patterns differ across the three configurations. The Diagonal model exhibits a gradual decrease in accuracy as noise increases, indicating comparatively stable robustness under the evaluated perturbation conditions. In contrast, the Full-shared model shows substantially greater degradation, suggesting that the shared training and regularization configuration is not well adapted to the denser cross-term representation. The Full-tuned model markedly mitigates this degradation through configuration-specific adjustment, although its noisy-condition performance remains lower than that of the Diagonal model. Overall, these results indicate that the observed robustness differences depend jointly on the polynomial representation and its associated training and regularization configuration, rather than on the representation type alone.

These results indicate that robustness is jointly determined by representation structure and regularization strategy, rather than representation type alone.

TABLE A-III.
COMPLEXITY AND TRAINING COST OF DIAGONAL AND FULL POLYNOMIAL IMPLEMENTATIONS

Configuration	Polynomial terms	Operator Params	Total Params (K)	FLOPs (M)
Diagonal PolyConv	5	164	152	37.82
Full local polynomial	70	2,240	155	38.13

TABLE A-IV
TRAINING, REGULARIZATION, AND TUNING SETTINGS OF DIAGONAL AND FULL POLYNOMIAL MODELS

Setting	Diagonal	Full-shared	Full-tuned
Polynomial form	Diagonal	Full local	Full local
Initialization	Fourth-order-priority	Fourth-order-priority	Equal-scale
Optimizer	Adam	Adam	Adam
Learning rate	0.001	0.001	0.001
Batch size	32	32	32
Early stopping	None	None	None
Orthogonality weight	0.02	0.02	0
Energy weight	0.10	0.10	0
Smoothness weight	0.001	0.001	0
Classifier L2	0.1	0.1	0.1
Dropout	0.5	0.5	0.5
Selection criterion	Fixed proposed setting	Fixed shared setting	Validation performance

TABLE A-V.
REPEATED-RUN ACCURACY OF DIAGONAL AND FULL POLYNOMIAL IMPLEMENTATIONS.

Model	Clean (%)	SNR20 (%)	SNR10 (%)
Diagonal Volterra form	99.87 ± 0.41	99.85 ± 0.37	92.60 ± 1.21
Full Shared Volterra form	71.42 ± 1.56	51.16 ± 5.72	26.14 ± 7.24
Full Tuned Volterra form	98.35 ± 0.65	91.60 ± 2.38	60.54 ± 4.94

TABLE A-VI.
NOISE SENSITIVITY INDEX OF DIFFERENT POLYNOMIAL ORDERS.

Order	NSI
1 st order	0.6188 ± 0.0526
2 nd order	0.6121 ± 0.0647
3 rd order	1.3306 ± 0.0709
4 th order	1.4886 ± 0.1655
5 th order	2.3239 ± 0.2211

C. Polynomial-Order Selection and Noise Sensitivity

Polynomial-order ablation (orders 1–5) is conducted under identical training settings. This study evaluates how increasing polynomial expressiveness affects both classification performance and stability.

The results indicate that performance does not increase monotonically with order. Instead, higher-order terms introduce additional nonlinear interactions that may improve expressiveness but also increase sensitivity to noise and optimization instability. The fourth-order configuration provides the best empirical balance under the current dataset, achieving a trade-off between representational richness and robustness.

In addition to the preliminary order-selection experiment, we further designed a noise sensitivity index (NSI) to quantitatively evaluate the perturbation sensitivity of each

polynomial-order component. The NSI of the p^{th} order component is defined as:

$$\text{NSI}_p = \frac{|\Phi_p(x + \eta) - \Phi_p(x)|_F}{|\Phi_p(x)|_F + \varepsilon} \quad \text{A(1)}$$

where x denotes the original input signal, η denotes the injected Gaussian noise, $\Phi_p(\cdot)$ denotes the p^{th} order PolyConv feature response, $|\cdot|_F$ is the Frobenius norm, and ε is a small constant used for numerical stability. A larger NSI value indicates that the corresponding polynomial-order response is more sensitive to noise perturbation. The NSI results are summarized in Table A-VI.

D. Implementation and Hyperparameter Details

The full implementation settings of the proposed PTCN are summarized in Table A-VII, including input configuration, PolyConv structure, constraints, and TCN/MHSA settings.

These settings are fixed across all experiments unless otherwise specified, ensuring that performance differences arise from model structure rather than training protocol variations.

All baseline architectures and their configurations are listed in Table A-VIII, using a unified training protocol for fair comparison.

TABLE A-VII.
HYPERPARAMETER SETTINGS OF PROPOSED MODELS

Category	Hyperparameter	Value	Category	Hyperparameter	Value
Training	Input size	256×1	Training	Optimizer	Adam
	Number of classes	7		Learning rate	0.001
	Iteration	655		Loss function	Categorical cross-entropy
	Batch size	32		Random seed	42
PolyConv	Maximum order	4	PolyConv	Padding	Same
	Polynomial type	Diagonal		activation	ReLU
	Number of filters	32		Kernel size / stride	4 / 4
Constraints	λ_{orth}	0.02	Downstream Network	TCN dilation rates	1, 2, 4
	λ_{energy}	0.10		TCN kernel size	4
	λ_{smooth}	0.001		Classifier hidden units	128
Downstream Network	Classifier L2	0.1	Downstream Network	Attention heads	8
	Dropout	0.5		Attention key dimension	16

TABLE A-VIII.
ARCHITECTURAL CONFIGURATIONS OF THE COMPARISON MODELS

Method	Complete Architecture and Hyperparameter Settings	
TCN	Architecture	Conv1D (16, k=64, s=8) → BN → ReLU → MaxPool (2) → Conv1D (32, k=3) → BN → ReLU → Conv1D (64, k=3) → BN → ReLU → Conv1D (128, k=3) → BN → ReLU → 4 × TCN residual block (dilations 1, 2, 4, 8; filters 64; kernel 3; causal padding; dropout 0.2) → GAP → Dense (128, ReLU) → Dropout (0.3) → Dense (7, Softmax)
	Settings	Adam (lr = 1×10^{-3}), categorical cross-entropy, batch size 32, 1 epoch.
CNN	Architecture	Conv1D (64, k=8) → BN → ReLU → MaxPool(2) → Conv1D (64, k=5) → BN → ReLU → MaxPool(2) → Conv1D (64, k=3) → BN → ReLU → MaxPool(2) → GAP → Dense (128, ReLU) → Dropout (0.3) → Dense (7, Softmax).
	Settings	Adam (lr = 1×10^{-3}), categorical cross-entropy, batch size 32, 1 epoch.
DCMS	Architecture	Stem Conv1D (16, k=7) → BN → ReLU → Dual-channel multi-scale blocks (Channel A: k ∈ {3,5,7,9}, f=32; Channel B: k ∈ {13,15,17,19}, f=32) → MaxPool (2) per channel → Second MS blocks → Concatenate → Conv1D (128, k=1) → BN → ReLU → GAP → Dense (128, ReLU) → Dropout (0.3) → Dense (7, Softmax).
	Settings	Adam (lr = 1×10^{-3}), categorical cross-entropy, batch size 32, 1 epoch.
RMFF	Architecture	5 parallel input branches Conv1D (2, k=1...5) → Concat → 3 Inception-style residual multi-scale fusion blocks (1×1 / 3×3 / 5×5 / MaxPool branches, residual connection) with MaxPool(2) + BN between blocks → Flatten → Dense (256, ReLU) → Dropout (0.3) → Dense (7, Softmax).
	Settings	Adam (lr = 1×10^{-3}), categorical cross-entropy, batch size 32, 1 epoch.
LSTM	Architecture	LSTM (128, return_sequences=True) → BN → LSTM (64) → Dense (128, ReLU) → Dropout (0.3) → Dense (7, Softmax).
	Settings	Adam (lr = 1×10^{-3}), categorical cross-entropy, batch size 32, 1 epoch.
V-BiLSTM	Architecture	AdaptiveMedianFilter1D(window=5) → WaveletVolterraExpansion (db6-like 3-level decomposition + third-order Volterra terms) → KroneckerFeaturePyramid (dilated convolutions at rates 2, 4, 6) → Conv1D (64, k=5) → BN → ReLU → MaxPool(2) → Conv1D(64, k=3) → BN → ReLU → Conv1D (128, k=3) → BN → ReLU → GAP → Dense (128, ReLU) → Dropout (0.5) → Dense (7, Softmax)
	Settings	Adam (lr = 1×10^{-3}), categorical cross-entropy, batch size 32, 1 epoch.
ROAT	Architecture	ROATenvelopeSpectrum (Hilbert orthogonal axis → pointwise rectified envelope → real FFT magnitude spectrum → log1p) → BN → Conv1D (32, k=5) → BN → ReLU → MaxPool(2) → Conv1D (64, k=3) → BN → ReLU → MaxPool(2) → Conv1D (64, k=3) → BN → ReLU → GAP → Dense (64, ReLU) → Dropout (0.3) → Dense (7, Softmax).
	Settings	Adam (lr = 1×10^{-3}), categorical cross-entropy, batch size 32, 1 epoch.

To ensure reproducibility, all baseline models are re-implemented or directly adopted from official sources when available, and trained under identical preprocessing, segmentation, and optimization settings. This guarantees that performance differences reflect architectural differences rather than implementation bias.

E. Edge-Deployment Evaluation and Measurement Protocol

The edge-deployment experiments were used to characterize the computational feasibility of the proposed PTCN on RK3576, NVIDIA Jetson Orin NX, and Raspberry Pi 4B. The platform accuracies reported in the main manuscript correspond to the 20 dB Gaussian-noise condition. Although the same underlying held-out test samples and evaluation protocol were used, Gaussian noise was generated separately during each platform evaluation rather than using one identical archived noisy tensor set. Consequently, the numerical inputs were not strictly identical across platforms. The small differences in classification accuracy are therefore not interpreted as hardware-dependent performance differences, but only as comparable deployment results under the tested noisy condition.

For the energy measurements, the device supply current was obtained from the voltage drop across a series sampling resistor; no standalone current sensor was used. The resistor voltage was acquired using an NI USB-4431 in DC-coupled mode at 1 kS/s, corresponding to an effective input bandwidth of approximately 490 Hz. The principal measurement settings are summarized in Table A-IX.

TABLE A-IX.
ENERGY MEASUREMENT CONFIGURATION AND INSTRUMENT SPECIFICATIONS

Item	Measurement information
Current sensing method	Voltage drops across a series sampling resistor
Data-acquisition module	NI USB-4431, National Instruments
Input coupling	DC
Sampling frequency	1 kS/s
Effective input bandwidth	Approximately 490 Hz

The current was derived from the measured voltage drop and the known sampling resistance according to:

$$I(t) = \frac{V_R(t)}{R_s} \quad A(2)$$

where $V_R(t)$ denotes the measured voltage across the sampling resistor and R_s denotes its resistance. Under the fixed DC supply condition, the instantaneous device power is represented as:

$$P(t) = V_s I(t) \quad A(3)$$

where V_s denotes the device supply voltage. The quiescent power P_q was obtained from the corresponding non-inference operating state. The incremental energy associated with the complete 100-inference execution block was then calculated as:

$$E_{100} = \int_{t_s}^{t_e} [P(t) - P_q] dt \quad A(4)$$

where t_s and t_e denote the beginning and end of the 100-inference measurement interval.

For the 1 kS/s discrete acquisition, the same quantity can be expressed as:

$$E_{100} \approx \sum_{j=1}^M (P_j - P_q) \Delta t, \quad \Delta t = 1 \text{ ms} \quad A(5)$$

The reported mean incremental energy per inference was subsequently obtained by $E_{\text{inf}} = \frac{E_{100}}{100}$. When the complete inference interval is represented using its segment-averaged operational power, the above expression becomes

$$E_{\text{inf}} = \frac{(\bar{P}_{\text{operational}} - P_q)(t_e - t_s)}{100}. \quad A(6)$$

Because the average inference time over the same 100 executions is $T_{\text{avg}} = \frac{t_e - t_s}{100}$, this is equivalent to the concise expression previously used in the manuscript.

$$E_{\text{inf}} = (\bar{P}_{\text{operational}} - P_q) T_{\text{avg}}. \quad A(7)$$

Thus, the reported value should be interpreted as a device-level incremental energy estimate averaged over a continuous 100-inference measurement block, rather than as a high-resolution measurement of one isolated inference transient.

The timer was started immediately before the model-inference call and stopped immediately after completion of the model-output calculation. The input data were prepared before the timed inference call. Therefore, the reported latency excludes physical signal acquisition, preprocessing, data transfer, and postprocessing.

At the 4 kHz signal-sampling frequency used in this study, acquiring one 256-point input segment requires $T_{\text{acq}} = 64 \text{ ms}$. Accordingly, the reported edge-device latency values characterize the computational time required to process an already available 256-point signal segment and should not be interpreted as end-to-end diagnostic latency. Because signal acquisition, preprocessing, transfer, inference, and postprocessing were not measured as one synchronized end-to-end process, no end-to-end diagnostic latency is reported.

RK3576 and Raspberry Pi 4B were evaluated using CPU-side PyTorch inference, with no NPU acceleration used on RK3576. NVIDIA Jetson Orin NX used an FP16 GPU-side TorchScript deployment file without INT8 quantization. Therefore, the smaller serialized model size reported for Jetson Orin NX reflects the platform-specific FP16 deployment representation rather than a different trained PTCN architecture.